



Information systems for Big Data

Prof. G. Fenza gfenza@unisa.it

WordCloud (Wrong Solution)



```
#3 TagCloud dei termini|
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

pipeline=[
{"$group":{"_id":"$text","count":{"$sum":1}}},
{"$sort":{"count":-1}}
]
lst=list(tweet_coll.aggregate(pipeline))
comment_words=''
stopwords=set(STOPWORDS)

for valore in lst:
    tokens=valore['_id']
    comment_words+=tokens+" "

tagc=WordCloud(width=800, height=800, background_color='white', stopwords=stopwords, min_font_size=10).generate(comment_words)
plt.figure(figsize=(8,8), facecolor=None)
plt.imshow(tagc)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

WordCloud (Right Solution)



- Mongo

```
[{ '$project' : {'Parole' : { '$regexFindAll': { 'input': '$text' , 'regex': r"\b\S+\b", 'options': "i" } } }},  
{ '$unwind' : "$Parole" },  
{ '$group': {  
    '_id': '$Parole.match',  
    'Conteggio': { '$sum': 1 }  
}},  
{ '$sort': { 'Conteggio': -1 } } ]
```

```
d = collection_restaurants.aggregate(pipeline)
```

WordCloud (Right Solution)



```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from PIL import Image
import numpy as np
font="UntitledFolder/CHICKENPie.ttf"
image= np.array(Image.open('UntitledFolder/tiktok.jpg'))

wc=WordCloud(width=400, height=800,font_path=font, background_color='black',
             stopwords=stopwords, min_font_size=1,
             mask=image).generate_from_frequencies(d)

plt.figure(figsize=(8,8), facecolor=None)
plt.imshow(wc)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```



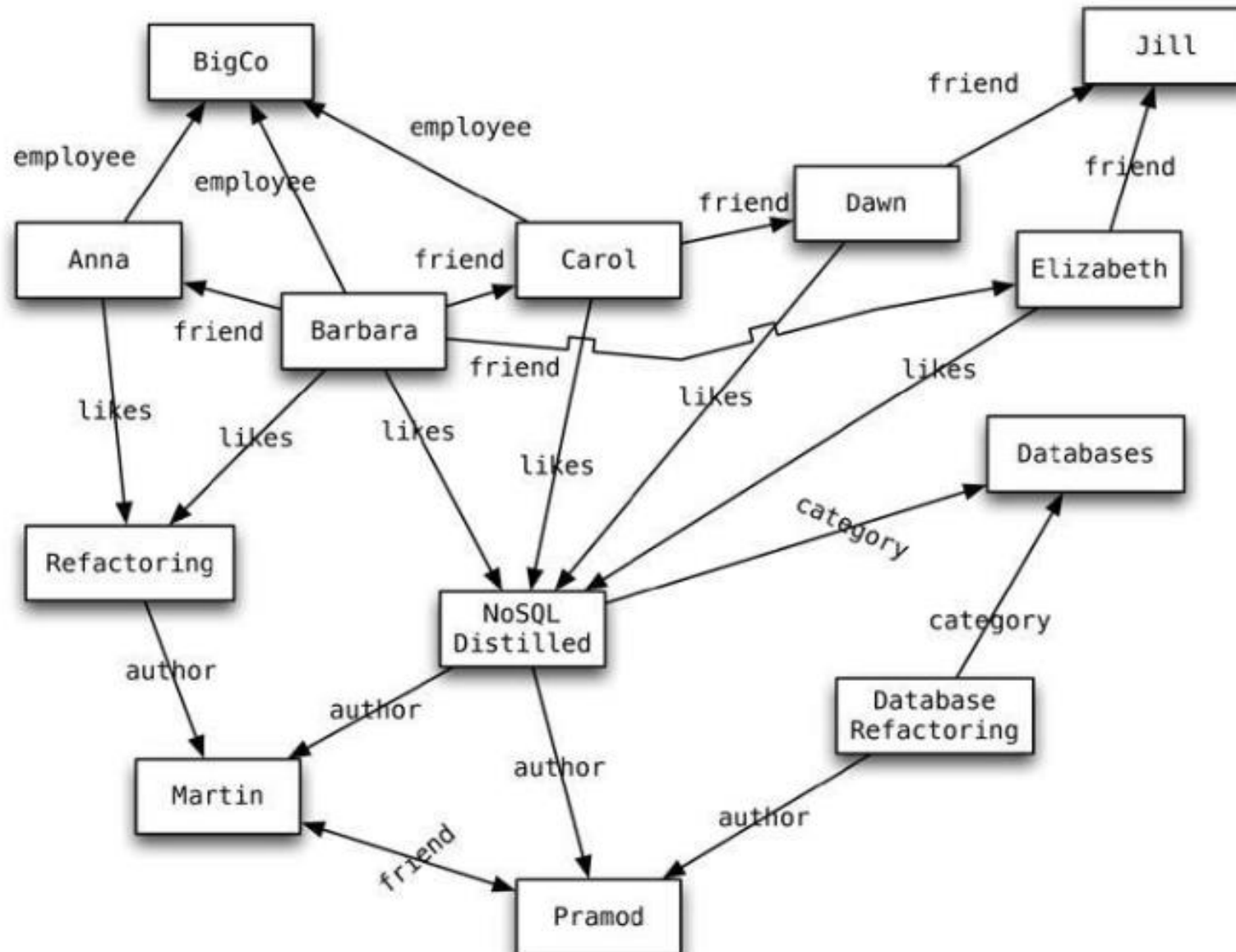


6th Lesson - NoSQL

Graph Database

- Graph databases store all of their information in nodes (vertices) and edges.
- Graph traversal is how you “query” the database.
- Relationship information about nodes is stored in the edges.
- Graph databases are motivated by a different frustration with relational databases
 - Complex relationships require complex join
- Goal:
 - Capture data consisting of complex relationships
 - Data naturally modelled as graphs
 - Examples: Social networks, Web data, product preferences

Graph DB Example



Data model of graph databases



- Basic characteristic: nodes connected by edges (also called arcs)
- Beyond this: a lot of variation in data models
 - FlockDB is simply nodes and edges with no mechanism for additional attributes
 - Neo4J stores Java objects to nodes and edges in a schema-less fashion
 - Infinite Graph stores Java objects, which are subclasses of built-in types, as nodes and edges
- Queries
 - Navigation through the network of edges
 - You do need a starting place
 - Nodes can be indexed by an attribute such as ID

Graph vs Relational Databases



- Relational databases implement relationships using foreign keys joins require to navigate around and can get quite expensive
- Graph databases
 - make traversal along the relationships very cheap
 - performance is better for highly connected data
 - shift most of the work from query time to insert time
 - good when querying performance is more important than insert speed

Graph vs Aggregate-oriented Databases



- Very different data models
- Aggregate-oriented databases
 - distributed across clusters
 - simple query languages no ACID guarantees
- Graph databases
 - more likely to run on a single server
 - graph-based query languages
 - transactions maintain consistency over multiple nodes and edges



db4o



6th Lesson – Hands on NoSQL

- GraphDB: NEO4J

Social Network “path exists” Performance



- Experiment:
 - ~1k persons
 - Average 50 friends per person
 - `pathExists(a,b)` limited to depth 4
 - Caches warm to eliminate disk IO

	# persons	query time
Relational database	1000	2000ms

Social Network “path exists” Performance



- Experiment:
 - ~1k persons
 - Average 50 friends per person
 - `pathExists(a, b)` limited to depth 4
 - Caches warm to eliminate disk IO

	# persons	query time
Relational database	1000	2000ms
Neo4j	1000	2ms

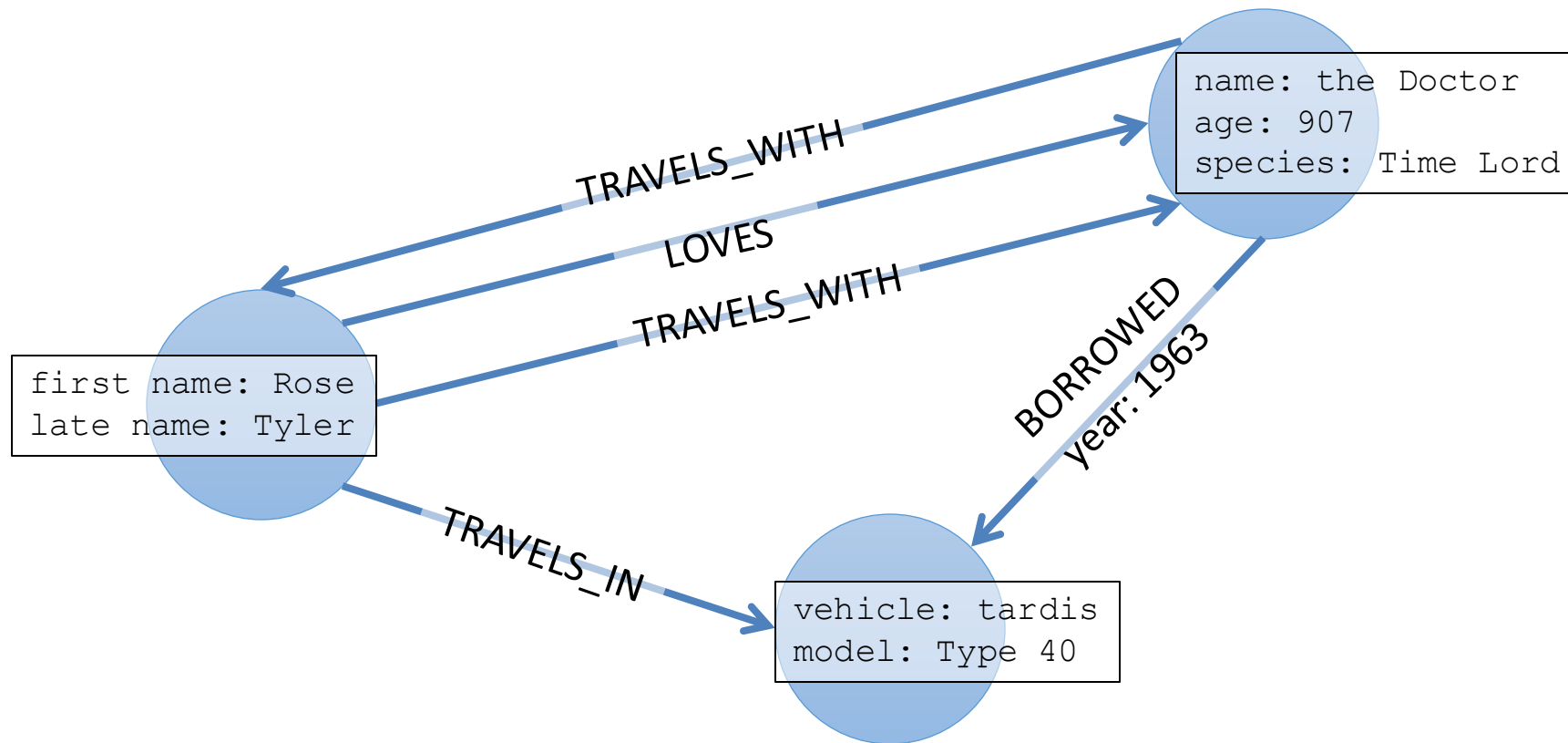
Social Network “path exists” Performance



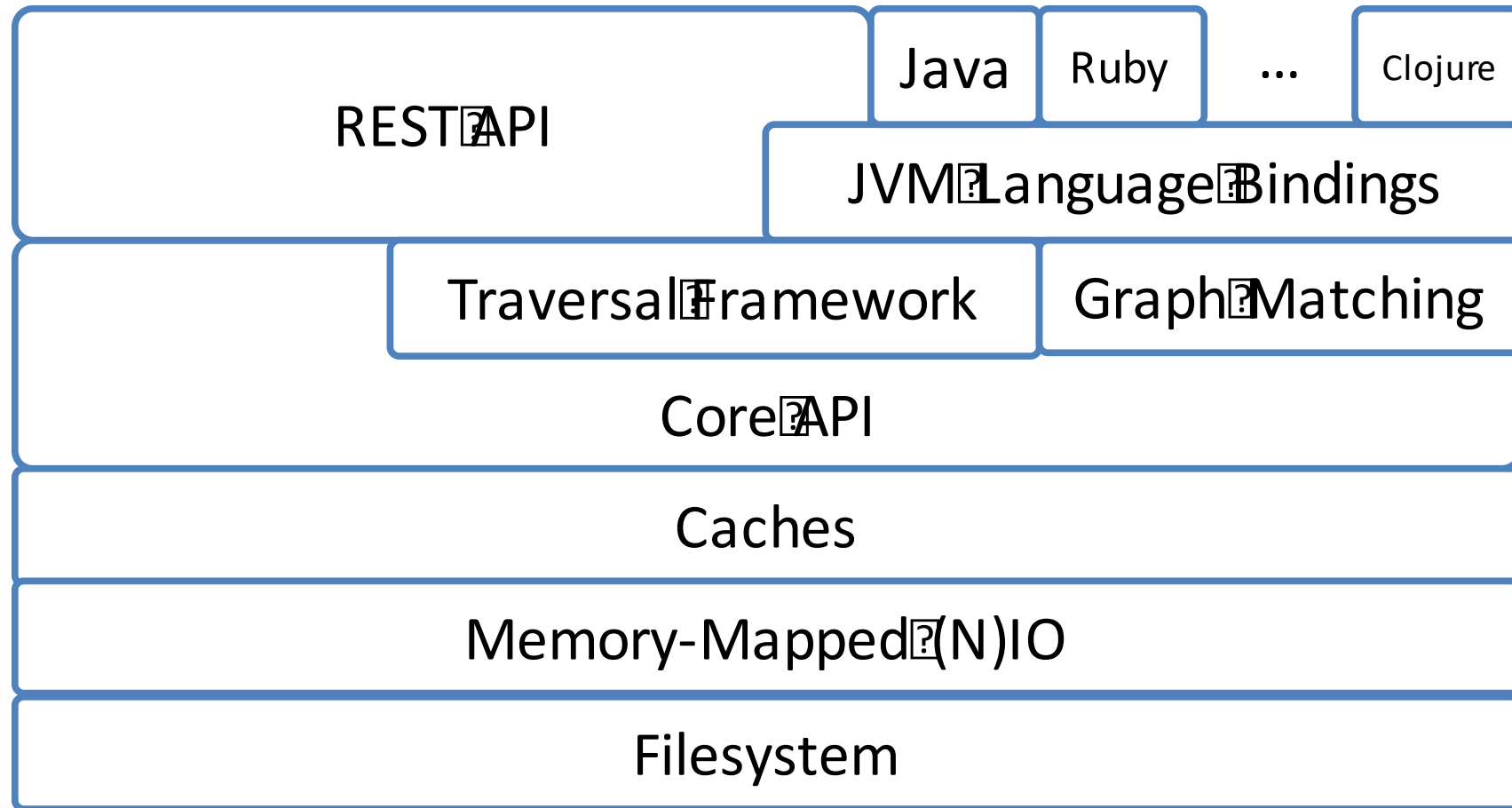
- Experiment:
 - ~1k persons
 - Average 50 friends per person
 - `pathExists(a, b)` limited to depth 4
 - Caches warm to eliminate disk IO

	# persons	query time
Relational database	1000	2000ms
Neo4j	1000	2ms
Neo4j	1000000	2ms

Property Graph Model



Neo4j Logical Architecture



- Deals with graphs in terms of their fundamentals:
 - Nodes
 - ❖ Properties
 - KV Pairs
 - Relationships
 - ❖ Start node
 - ❖ End node
 - ❖ Properties
 - KV Pairs

Indexing a Graph?



- Graphs are their own indexes!
- But sometimes we want short-cuts to well-known nodes
- Can do this in our own code
 - Just keep a reference to any interesting nodes
- Indexes offer more flexibility in what constitutes an “interesting node”

Cypher Query Language



- Existing Neo4j query mechanisms were not simple enough
 - Too verbose (Java API)
 - Too prescriptive (Gremlin)
- Declarative graph pattern matching language
 - “SQL for graphs”
 - Tabular results
- Cypher is evolving steadily
 - Syntax changes between releases
 - Graph Query Language for Neo4j
 - Aims to make querying simple

- SQL
 - Unable to express paths
 - Neo4j is schema/table free
- SPARQL
 - Designed for a different data model, triples
 - Namespaces
 - Properties as node

Cypher Query Language



- Is a query language for Neo4j Graph Database.
- Is a declarative pattern-matching language.
- Follows SQL like syntax.
- Syntax is very simple and in human readable format.
- Commands to perform Database operations.
- Clauses such as WHERE, ORDER BY, etc., to write very complex queries in an easy manner.
- Functions such as String, Aggregation. In addition to them, it also supports some Relationship Functions.

RDBMS vs Graph Database



RDBMS	Graph Database
Tables	Graphs
Rows	Nodes
Columns and Data	Properties and its values
Constraints	Relationships
Joins	Traversal

Read Clauses



Read Clauses	Usage
MATCH	This clause is used to search the data with a specified pattern.
OPTIONAL MATCH	This is the same as match, the only difference being it can use nulls in case of missing parts of the pattern.
WHERE	This clause is used to add contents to the CQL queries.
START	This clause is used to find the starting points through the legacy indexes.
LOAD CSV	This clause is used to import data from CSV files.

Write Clauses



Write Clause	Usage
CREATE	This clause is used to create nodes, relationships, and properties.
MERGE	This clause verifies whether the specified pattern exists in the graph. If not, it creates the pattern.
SET	This clause is used to update labels on nodes, properties on nodes and relationships.
DELETE	This clause is used to delete nodes and relationships or paths etc. from the graph.
REMOVE	This clause is used to remove properties and elements from nodes and relationships.
FOREACH	This class is used to update the data within a list.
CREATE UNIQUE	Using the clauses CREATE and MATCH, you can get a unique pattern by matching the existing pattern and creating the missing one.
Importing CSV files with Cypher	Using Load CSV you can import data from .csv files.

General Clauses



General Clauses	Usage
RETURN	This clause is used to define what to include in the query result set.
ORDER BY	This clause is used to arrange the output of a query in order. It is used along with the clauses RETURN or WITH .
LIMIT	This clause is used to limit the rows in the result to a specific value.
SKIP	This clause is used to define from which row to start including the rows in the output.
WITH	This clause is used to chain the query parts together.
UNWIND	This clause is used to expand a list into a sequence of rows.
UNION	This clause is used to combine the result of multiple queries.
CALL	This clause is used to invoke a procedure deployed in the database.

Data Types



CQL Data Type	Usage
Boolean	It is used to represent Boolean literals: true, false.
byte	It is used to represent 8-bit integers.
short	It is used to represent 16-bit integers.
int	It is used to represent 32-bit integers.
long	It is used to represent 64-bit integers.
float	It is used to represent 32-bit floating-point numbers.
double	It is used to represent 64-bit floating-point numbers.
char	It is used to represent 16-bit characters.
String	It is used to represent Strings.

Functions



CQL Functions	Usage
String	They are used to work with String literals.
Aggregation	They are used to perform some aggregation operations on CQL Query results.
Relationship	They are used to get details of relationships such as startnode, endnode, etc.

Operators



Boolean Operators	Description
=	It is a Neo4j CQL "Equal To" operator.
< >	It is a Neo4j CQL "Not Equal To" operator.
<	It is a Neo4j CQL "Less Than" operator.
>	It is a Neo4j CQL "Greater Than" operator.
<=	It is a Neo4j CQL "Less Than Or Equal To" operator.
> =	It is a Neo4j CQL "Greater Than Or Equal To" operator.

Operators



Boolean Operators	Description
AND	It is a Neo4j CQL keyword to support AND operation. It is like SQL AND operator.
OR	It is a Neo4j CQL keyword to support OR operation. It is like SQL AND operator.
NOT	It is a Neo4j CQL keyword to support NOT operation. It is like SQL AND operator.
XOR	It is a Neo4j CQL keyword to support XOR operation. It is like SQL AND operator.

Operators

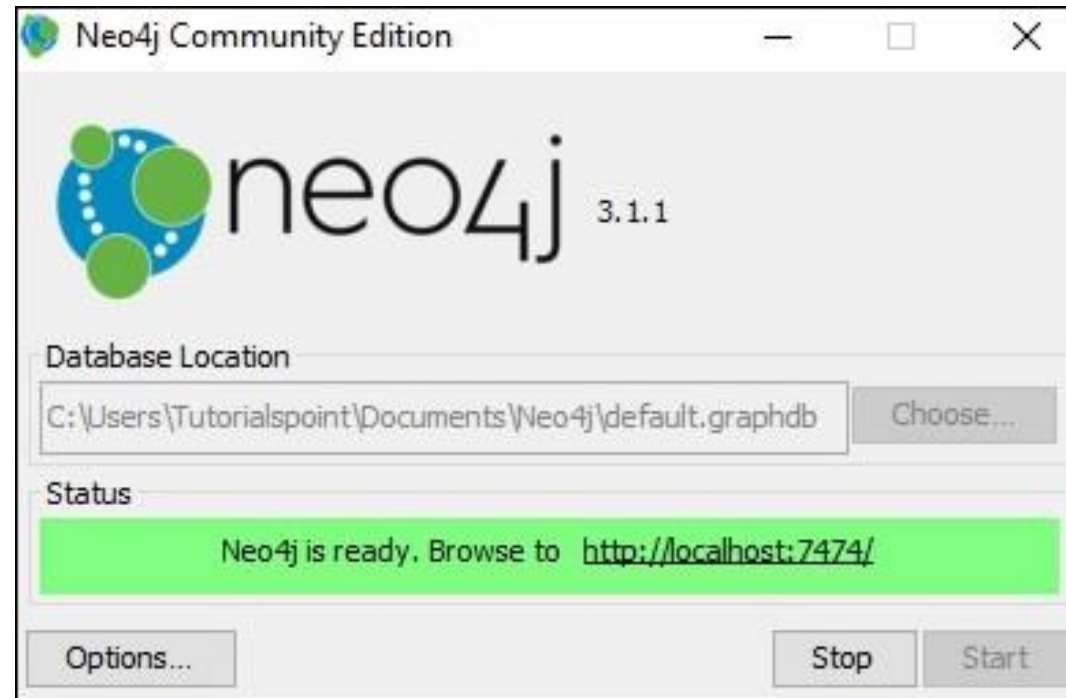


Type	Operators
Mathematical	+, -, *, /, %, ^
Comparison	+, < >, <, >, <=, >=
Boolean	AND, OR, XOR, NOT
String	+
List	+, IN, [X], [X.....Y]
Regular Expression	=-
String matching	STARTS WITH, ENDS WITH, CONSTRAINTS

NEO4J - Built-in browser app



- Open the Neo4j desktop App and start the Neo4j Server. Open the built-in browser app of Neo4j using the URL **<http://localhost:7474/>** as shown in the following screenshot.

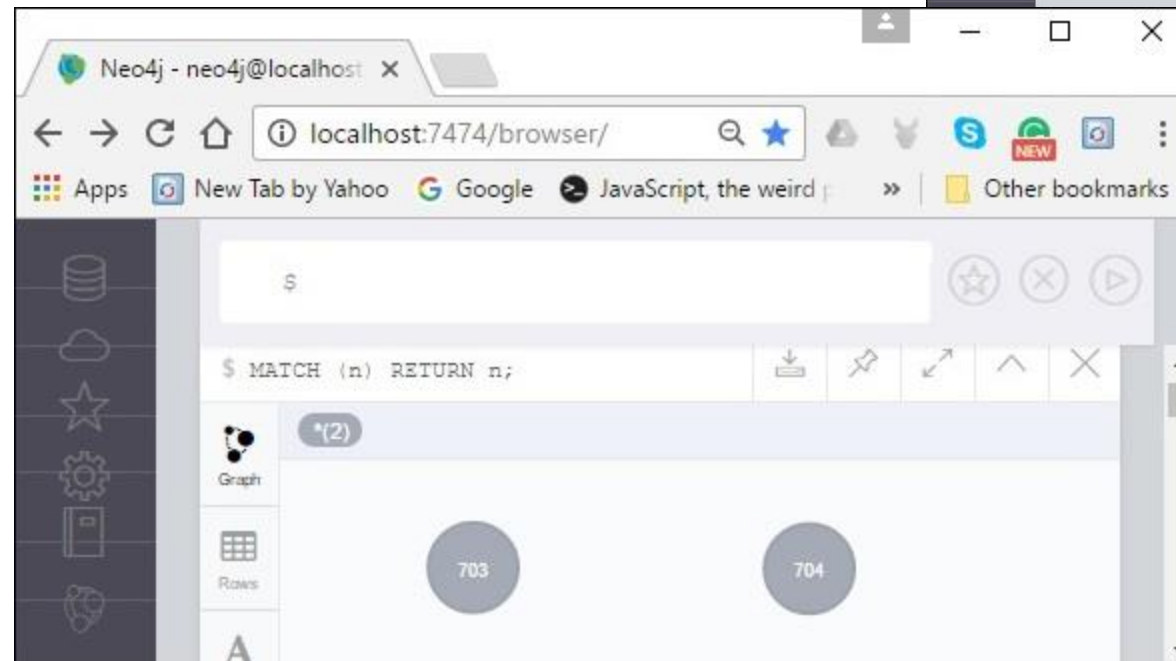
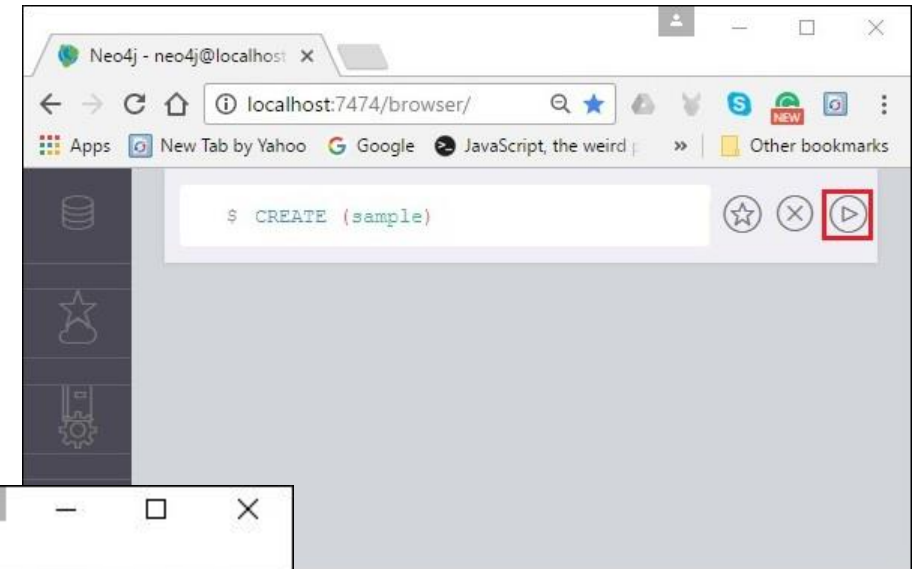


- `docker pull neo4j:latest`
- `docker run`
 - `--publish=7474:7474`
 - `--publish=7687:7687`
 - `--volume=$HOME/neo4j/data:/data`
 - `--volume=$HOME/neo4j/data:/import``neo4j:latest`

Write Clauses



- CREATE (sample)
- CREATE (node1),(node2)
- To verify: MATCH (n) RETURN n

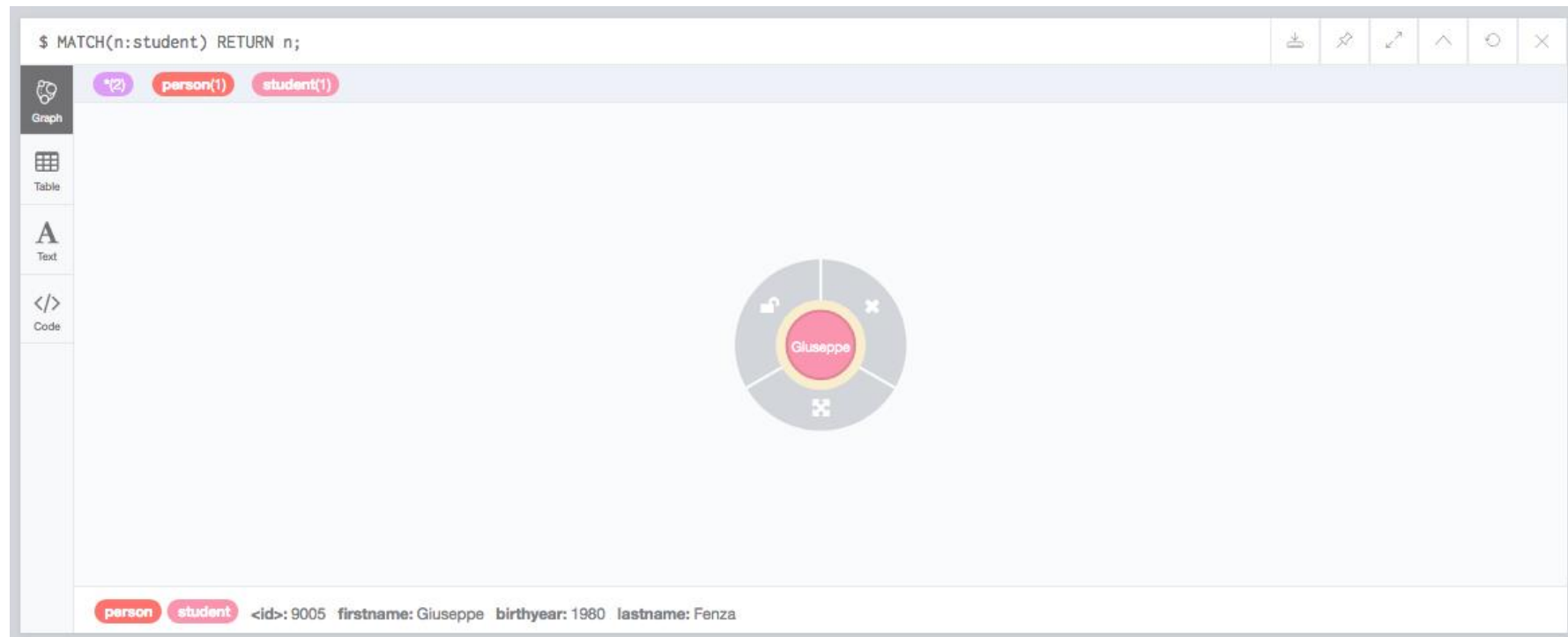


- Node with label CREATE (Giuseppe:person)
- Node with multiple labels CREATE (Giuseppe:person:student)
- Verify: MATCH (n) RETURN n

Write Clauses



- Node with label and properties
 - CREATE (Giuseppe:person:student {firstname: "Giuseppe", birthyear: 1980, lastname: "Fenza"})



Write Clauses



- **Creating Relationships**

- CREATE (Dhawan:player{name: "Shikar Dhawan", YOB: 1985, POB: "Delhi"})
- CREATE (Ind:Country {name: "India"})
- CREATE (Dhawan)-[r:BATSMAN_OF]->(Ind) **RETURN** Dhawan, Ind



- **Create Relationship Between Existing Nodes**

```
MATCH (a:player), (b:Country)
```

```
WHERE a.name = "Shikar Dhawan" AND b.name = "India"
```

```
CREATE (a)-[r: BATSMAN_OF]->(b)
```

```
RETURN a,b
```

- **Creating a Relationship with Label and Properties**

```
MATCH (a:player), (b:Country)
```

```
WHERE a.name = "Shikar Dhawan" AND b.name = "India"
```

```
CREATE (a)-[r:BATSMAN_OF {Matches:5, Avg:90.75}]->(b)
```

```
RETURN a,b
```

- **Setting a Property**

- MATCH (Dhawan:player{name: "shikar Dhawan", YOB: 1985, POB: "Delhi"})
- SET Dhawan.highestscore = 187 *(or labels **SET Dhawan: person:english**)*
- RETURN Dhawan

- **Removing a Property**

- MATCH (Jadeja:player {name: "Ravindra Jadeja", YOB: 1988, POB: "NavagamGhed"})
- SET Jadeja.POB = NULL
- RETURN Jadeja

- **Deleting a Particular Node**

- MATCH (Ishant:player {name: "Ishant Sharma", YOB: 1988, POB: "Delhi"})
- DETACH DELETE Ishant

- **Removing a Property**

- MATCH (Dhoni:player {name: "MahendraSingh Dhoni", YOB: 1981, POB: "Ranchi"})
- REMOVE Dhoni.POB (*or labels **REMOVE** Ishant:player:person*)
- RETURN Dhoni

- **Get All Nodes Using Match**
- **Match by label**
 - MATCH (n:player)
 - RETURN n
- **Match by Relationship**
 - MATCH (node:label)<-[: Relationship]-(n)
 - RETURN n

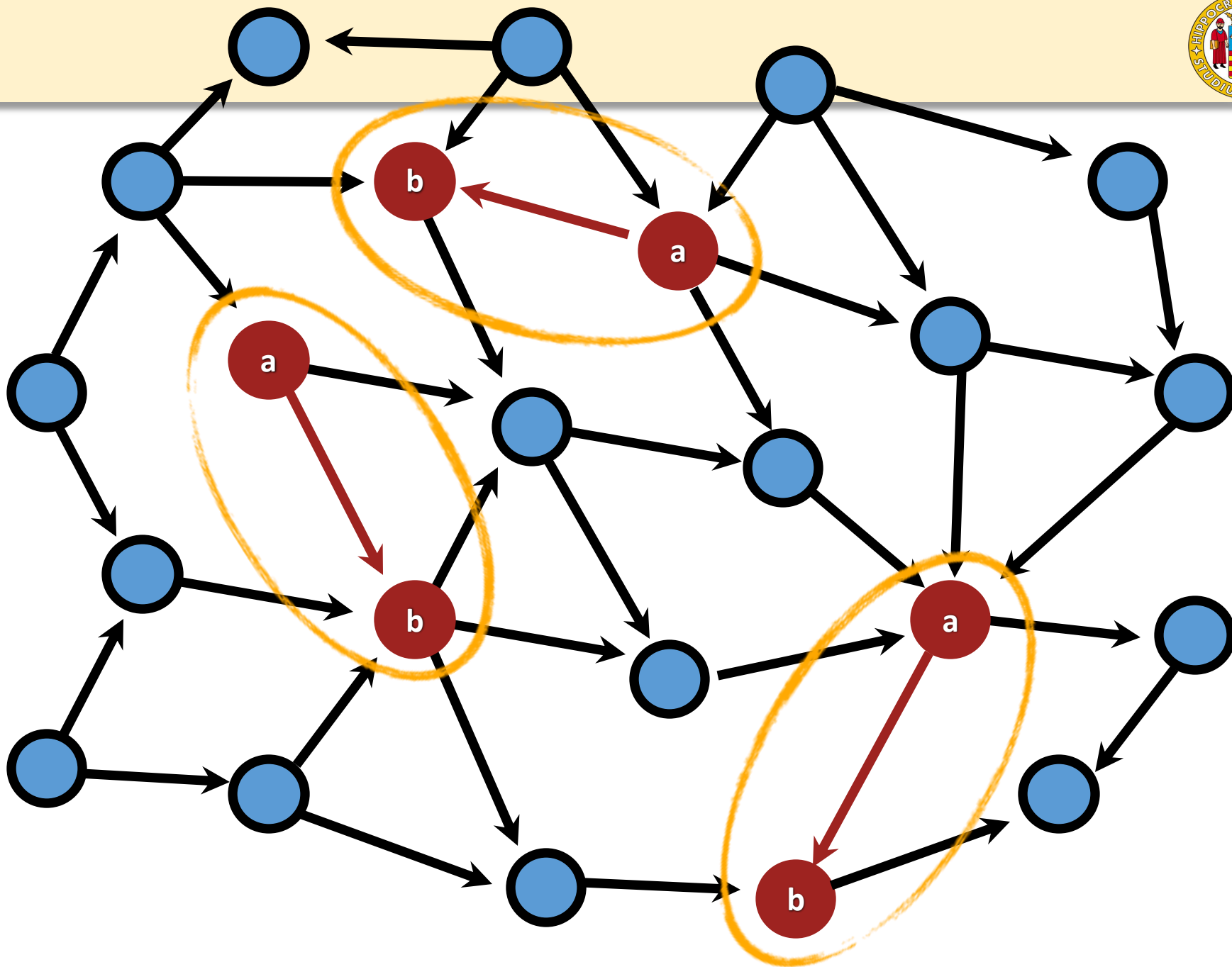
Two nodes, one relationship



MATCH (a)-->(b)

RETURN a, b;





MATCH (a)-->(b)
RETURN a, b;

Two nodes, one relationship



$(a) \rightarrow ()$

Two nodes, one relationship



```
MATCH (a)-->()  
RETURN a;
```



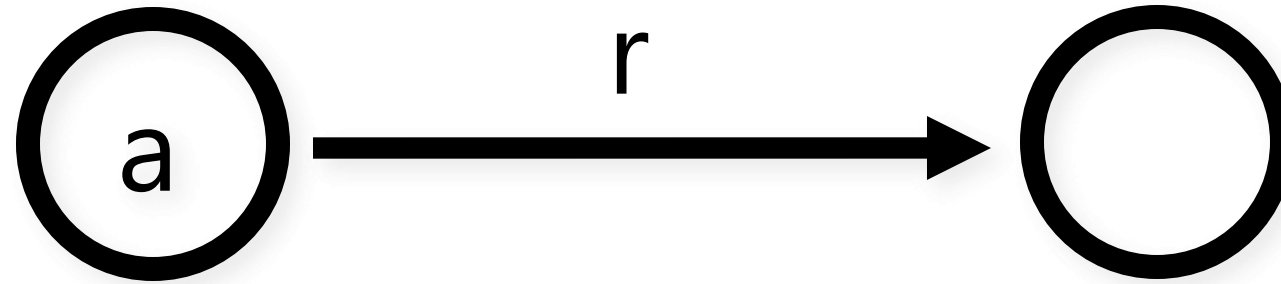
Two nodes, one relationship



```
MATCH (a)-->()  
RETURN a.name;
```



Two nodes, one relationship

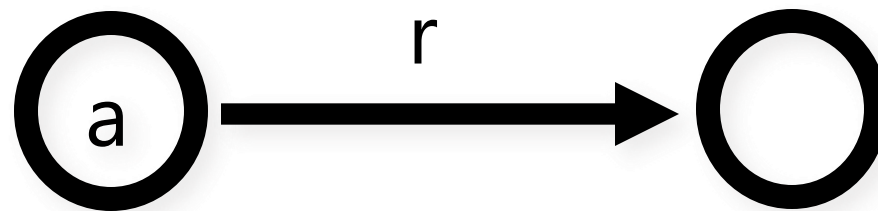


$(a) -[r] \rightarrow ()$

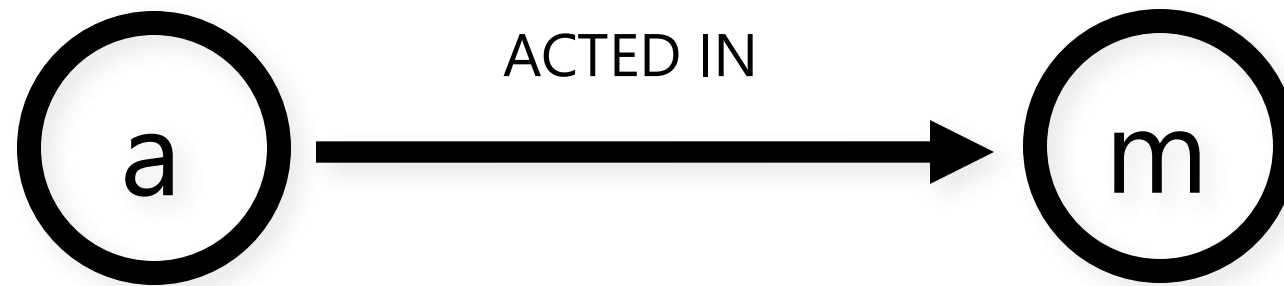
Two nodes, one relationship



```
MATCH (a)-[r]->()  
RETURN a.name, type(r);
```



Two nodes, one relationship



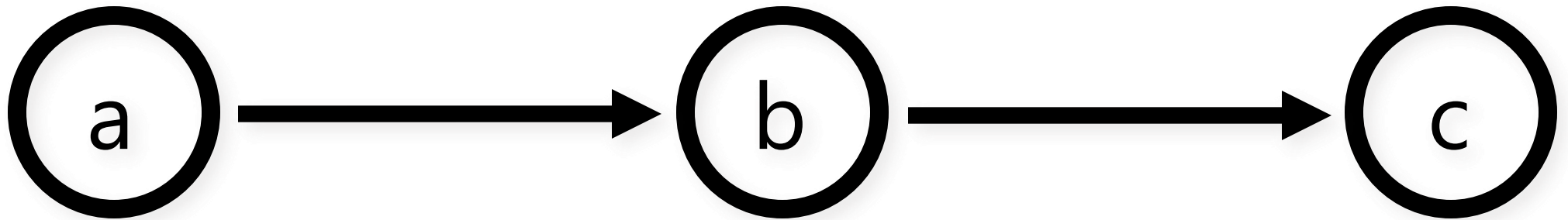
`(a) -[:ACTED_IN]-> (m)`

Two nodes, one relationship



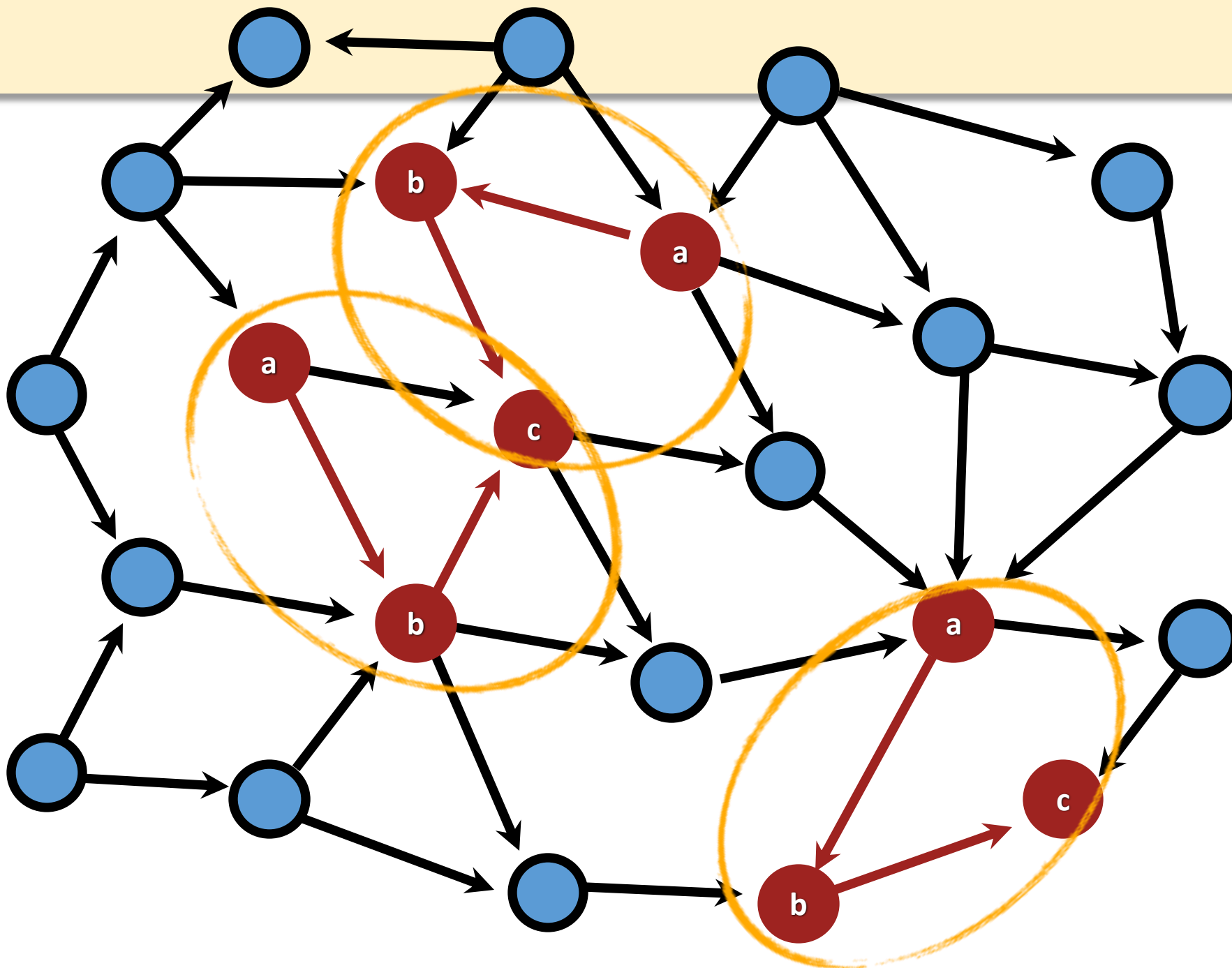
```
MATCH (a)-[:ACTED_IN]->(m)  
RETURN a.name, m.title;
```

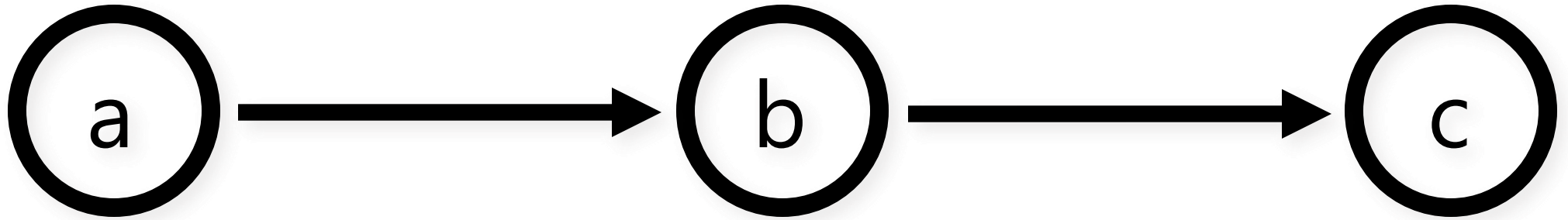




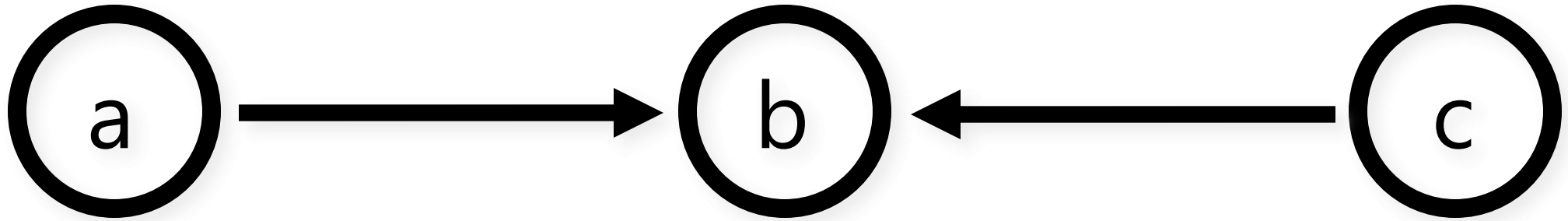
$(a) \dashrightarrow (b) \dashrightarrow (c)$

(a)-->(b)-->(c)



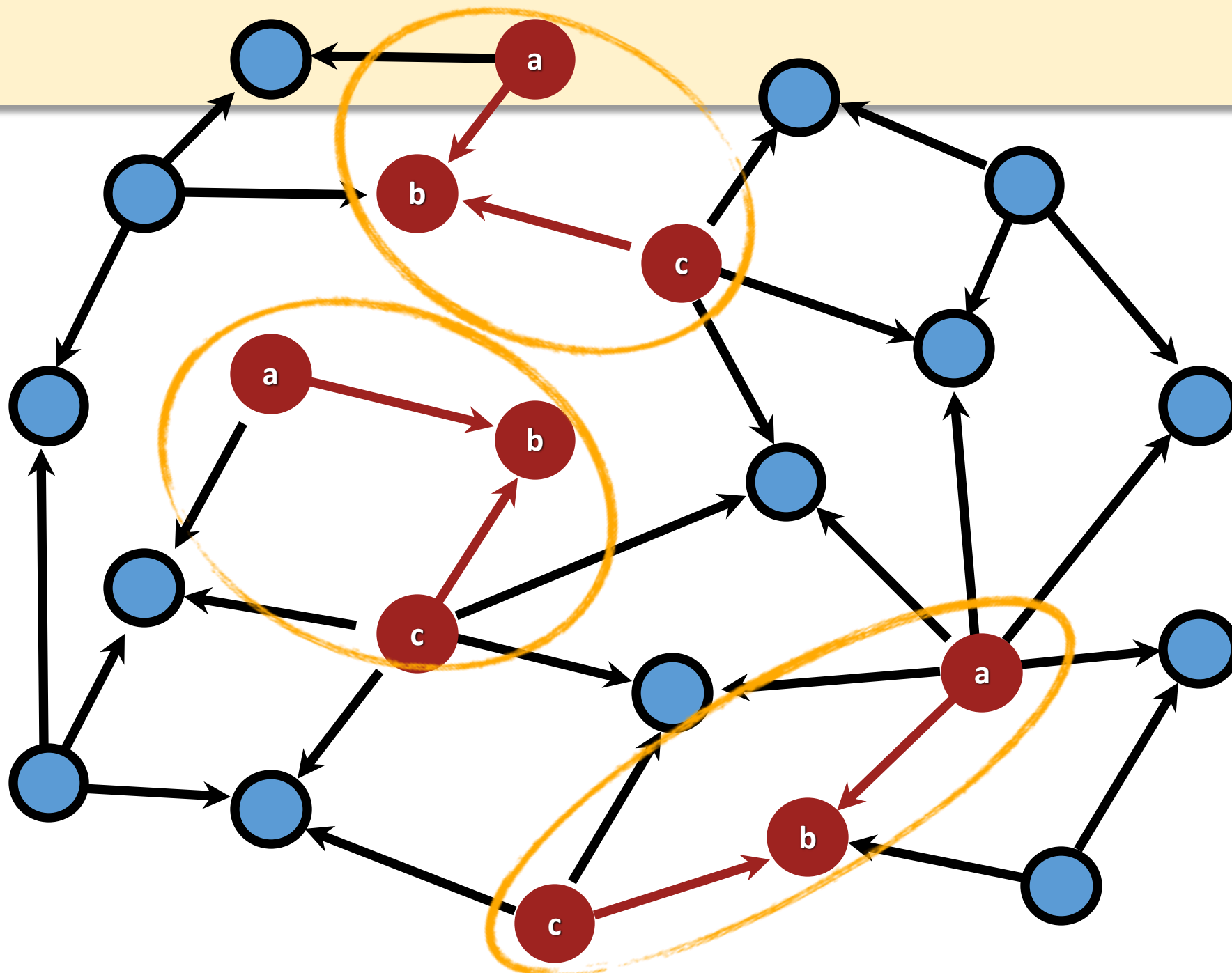


$(a) \dashrightarrow (b) \dashrightarrow (c)$

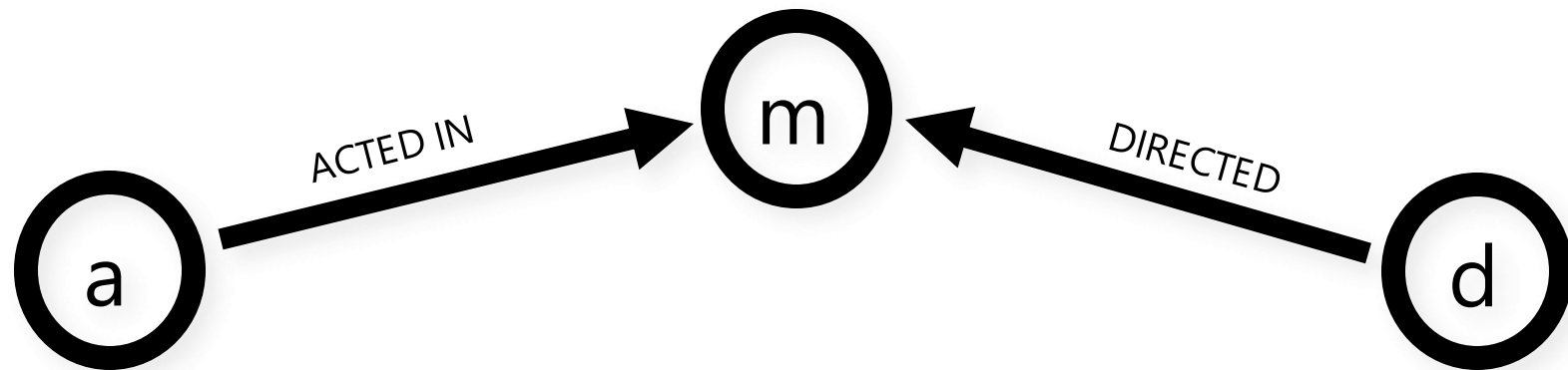


$(a) \dashrightarrow (b) \dashleftarrow (c)$

(a)-->(b)<--(c)

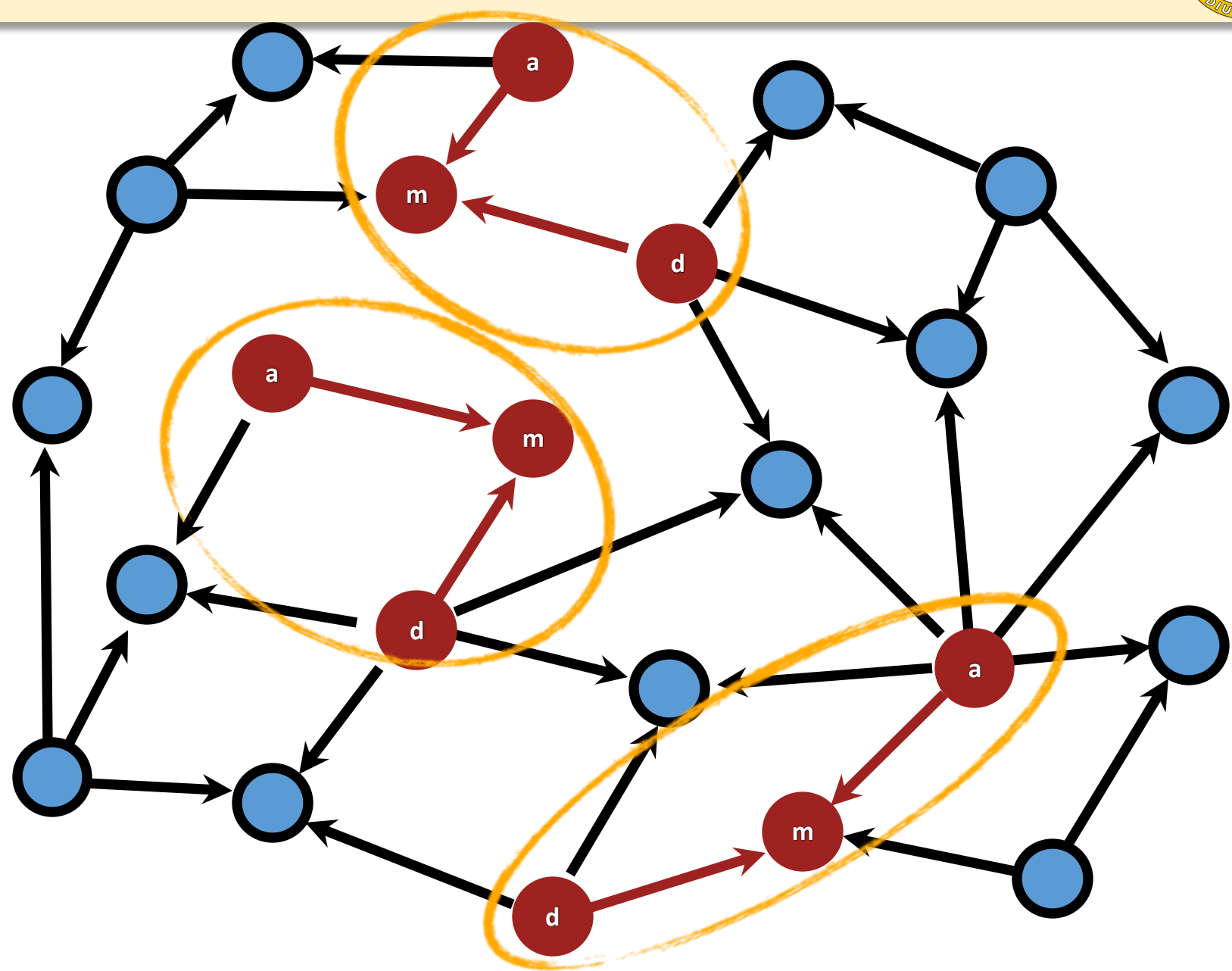


```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)  
RETURN a.name, m.title, d.name;
```



MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)

RETURN a.name, m.title, d.name;



```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)
```

```
RETURN a.name, m.title, d.name;
```

a.name	m.title	d.name
"Keanu Reeves"	"The Matrix"	"Andy Wachowski"
"Keanu Reeves"	"The Matrix Reloaded"	"Andy Wachowski"
"Noah Wyle"	"A Few Good Men"	"Rob Reiner"
"Tom Hanks"	"Cloud Atlas"	"Andy Wachowski"
...	...	...

```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)
RETURN a.name AS actor, m.title AS movie,
       d.name AS director;
```

actor	movie	director
"Keanu Reeves"	"The Matrix"	"Andy Wachowski"
"Keanu Reeves"	"The Matrix Reloaded"	"Andy Wachowski"
"Noah Wyle"	"A Few Good Men"	"Rob Reiner"
"Tom Hanks"	"Cloud Atlas"	"Andy Wachowski"
...	...	...

```
MATCH (a)-[:ACTED_IN]->(m), (m)<-[:DIRECTED]-(d)
```

```
RETURN a.name, m.title, d.name;
```



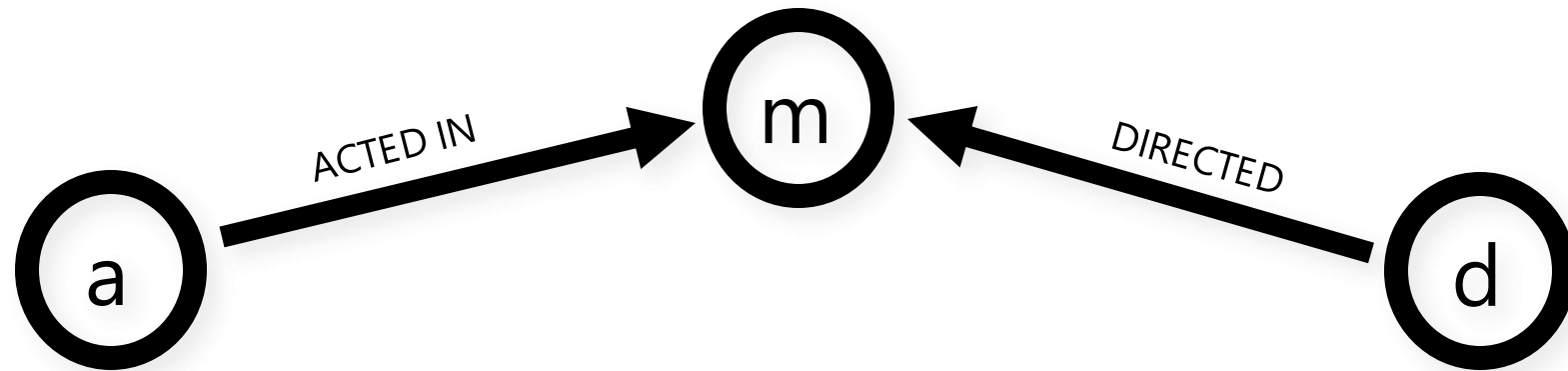
```
MATCH (a)-[:ACTED_IN]->(m), (d)-[:DIRECTED]->(m)
```

```
RETURN a.name, m.title, d.name;
```

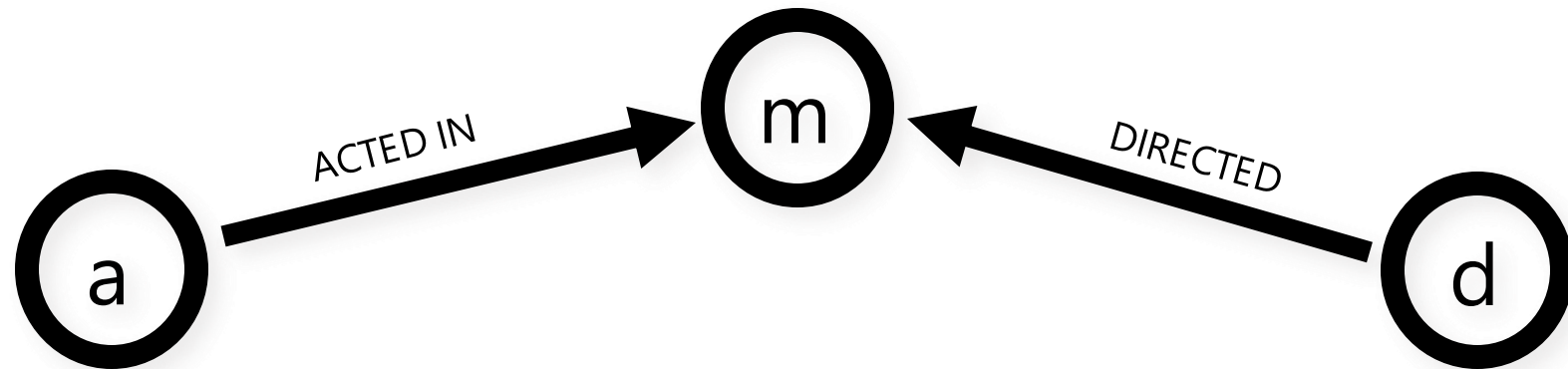


```
MATCH p=(a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)
```

```
RETURN p;
```



```
MATCH p=(a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)  
RETURN nodes(p);
```



```
MATCH p1=(a)-[:ACTED_IN]->(m), p2=(d)-[:DIRECTED]->(m)
```

```
RETURN p1, p2;
```



```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)
RETURN a.name, m.title, d.name;
```

a.name	m.title	d.name
"Keanu Reeves"	"The Matrix"	"Andy Wachowski"
"Keanu Reeves"	"The Matrix Reloaded"	"Andy Wachowski"
"Noah Wyle"	"A Few Good Men"	"Rob Reiner"
"Tom Hanks"	"Cloud Atlas"	"Andy Wachowski"
...	...	...

```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)
RETURN a.name, d.name, count(*);
```

a.name	d.name	count(*)
"Aaron Sorkin"	"Rob Reiner"	2
"Keanu Reeves"	"Andy Wachowski"	3
"Hugo Weaving"	"Tom Tykwer"	1
...	...	...

```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)
RETURN a.name, d.name, count(m);
```

a.name	d.name	count(m)
"Aaron Sorkin"	"Rob Reiner"	2
"Keanu Reeves"	"Andy Wachowski"	3
"Hugo Weaving"	"Tom Tykwer"	1
...	...	...

Which directors also acted in their movie?

```
MATCH (d)-[:DIRECTED]->(m)<-[:ACTED_IN]-(d)
```

```
RETURN d.name, m.title;
```

Sort & Limit



```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)
RETURN a.name, d.name, count(*) AS count
ORDER BY(count) DESC
LIMIT 5;
```

Aggregation

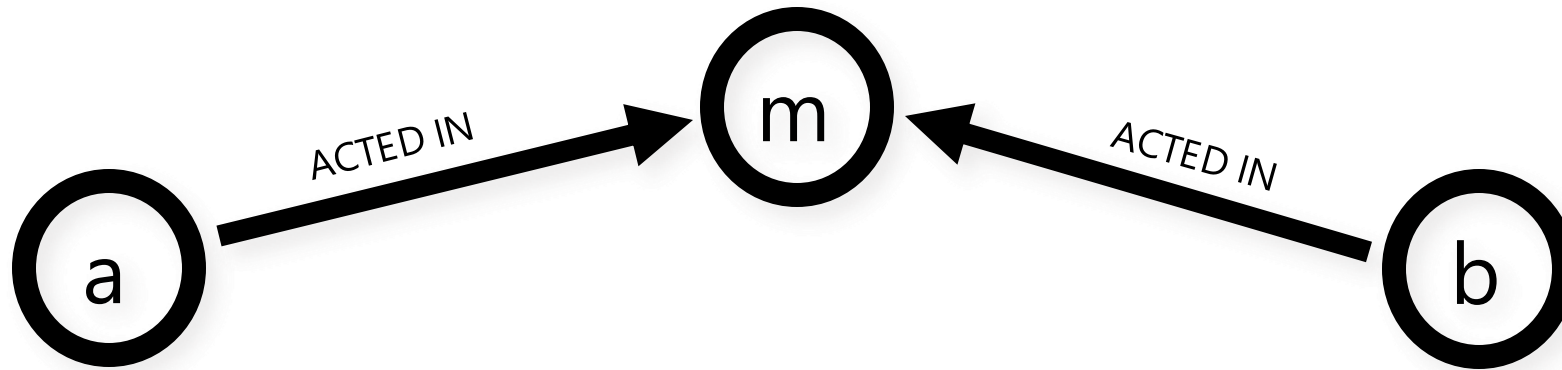


- count(x) - *add up the number of occurrences*
- min(x) - *get the lowest value*
- max(x) - *get the highest value*
- avg(x) - *get the average of a numeric value*
- collect(x) - *collected all the occurrences into an array*

Aggregation



```
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d)  
RETURN a.name, d.name, collect(m.title);
```



Find a specific node (all-node query)



```
MATCH (n)
```

```
WHERE exists(n.name) AND n.name = "Tom Hanks"
```

```
RETURN n;
```

WHERE - filter the results

exists(n.name) - the name property must exist

n.name = "Tom Hanks" - and have that value

A directed graph with 18 blue nodes and one red node labeled 't'. The nodes are interconnected by directed edges, forming a complex network structure. The red node 't' is located in the upper-middle part of the graph and has three outgoing edges. The blue nodes are distributed throughout the graph, with some having multiple incoming or outgoing edges, creating a dense network of connections.

Find a specific node (auto-index query)



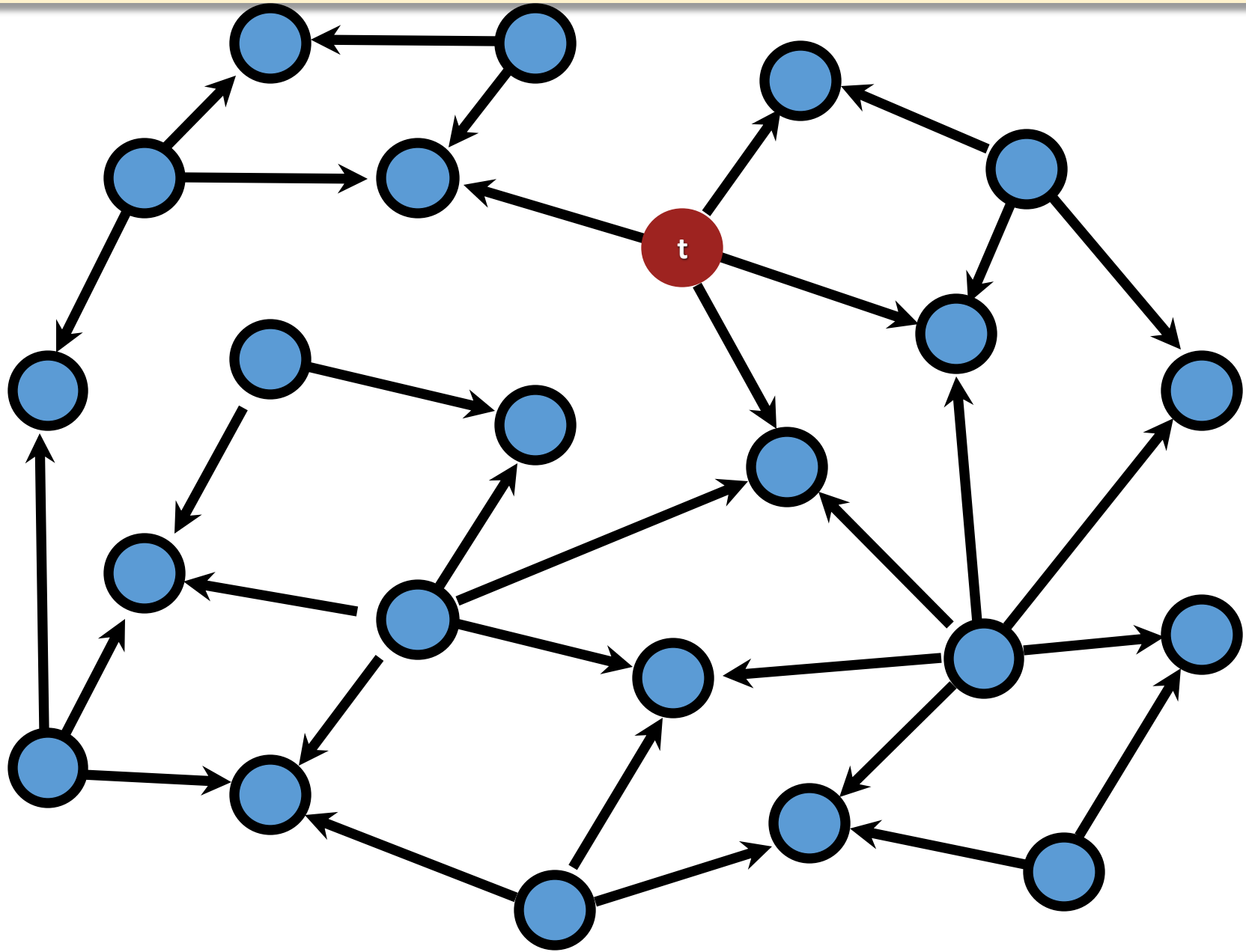
```
CREATE INDEX ON :Person(name);
```

```
MATCH (n:Person)
```

```
WHERE n.name = 'Tom Hanks'
```

```
RETURN n;
```

© 2014 Pearson Education, Inc. or its affiliate(s). All rights reserved.



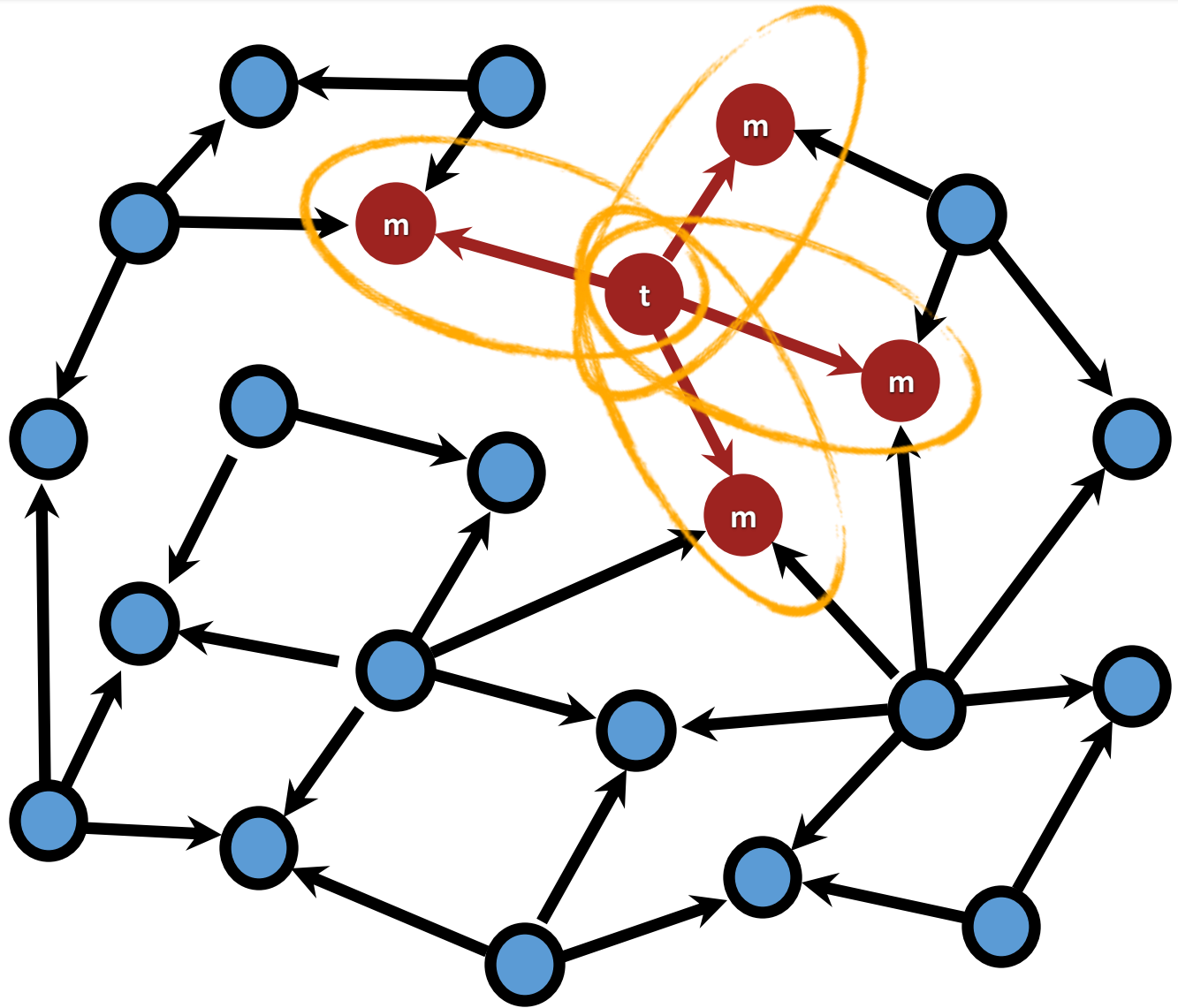
Start with a specific node (auto-indexed)



```
MATCH (tom:Person)
WHERE tom.name = 'Tom Hanks'
MATCH (tom)-[:ACTED_IN]->(movie)
RETURN movie.title;
```

(Movies featuring Tom Hanks)

```
MATCH (tom:Person)
WHERE tom.name = 'Tom Hanks'
MATCH (tom)-[:ACTED_IN]->(movie)
RETURN movie.title;
```





NEO4J – Exercise

- Q1: What is a shortest path (using any kind of relation) between Keanu Reeves and Tom Cruise?
- Q2: How long is the shortest path between Keanu Reeves and Tom Cruise?
- Q3: What are the shortest paths between people with first name Keanu and people with first name Tom?
- Q4: Which persons have a distance of length 6 to Kevin Bacon (the distance between two persons being the length of a shortest path between them)?
- Q5: How many persons are there with distance 6 to Kevin Bacon?

- Q6: What is the average number of persons with distance 6 to any person?
- Q7: What is the largest distance of any person to Kevin Bacon?
- Q8: What is the largest shortest-path distance between any two persons?
- Q9: What is the distribution of pairwise distances (i.e., for distance 1, 2, 3, ..., how many pairs of persons have that distance from each other)?
- Q10: In the co-actor graph, which 10 persons have the shortest average distance to any other person he/she is connected to?
- Q11: How many persons have persons co-acted with on average through distance 1, 2, 3, ...?

- Q1: What is a shortest path (using any kind of relation) between Keanu Reeves and Tom Cruise?
- Hint: *shortestPath((p1)-[*..15]-(p2))*
- Answer
 - MATCH (p1:Person { name: "Keanu Reeves" }),(p2:Person { name:"Tom Cruise" }), p
= shortestPath((p1)-[*..15]-(p2))
 - RETURN p
- Q2: How long is the shortest path between Keanu Reeves and Tom Cruise?
- Answer
 - MATCH (p1:Person { name: "Keanu Reeves" }),(p2:Person { name:"Tom Cruise" }), p
= shortestPath((p1)-[*..15]-(p2))
 - RETURN length(p)

- Q3: What are the shortest paths between people with first name Keanu and people with first name Tom?
- Answer
 - MATCH (p1:Person),(p2:Person), p = shortestPath((p1)-[*..15]-(p2))
 - WHERE p1.name STARTS WITH "Keanu" AND p2.name STARTS WITH "Tom"
 - RETURN p

- Q4: Which persons have a distance of length 6 to Kevin Bacon (the distance between two persons being the length of a shortest path between them)?
- Answer
 - MATCH (p1:Person),(p2:Person), p = shortestPath((p1)-[*..15]-(p2))
 - WHERE p1.name = "Kevin Bacon" AND p2.name <> "Kevin Bacon"
 - WITH p2,p
 - WHERE length(p) = 6
 - RETURN p2
- Q5: How many persons are there with distance 6 to Kevin Bacon?
- Answer
 - MATCH (p1:Person),(p2:Person), p = shortestPath((p1)-[*..15]-(p2))
 - WHERE p1.name = "Kevin Bacon" AND p2.name <> "Kevin Bacon"
 - WITH p2, p
 - WHERE length(p) = 6
 - RETURN count(*)

- Q6: What is the average number of persons with distance 6 to any person?
- Answer
 - MATCH (p1:Person),(p2:Person), p = shortestPath((p1)-[*..15]-(p2))
 - WHERE p1 <> p2
 - WITH p1, p
 - WHERE length(p) = 6
 - WITH p1, count(*) AS count
 - RETURN avg(count)
- Q7: What is the largest distance of any person to Kevin Bacon?
- Answer
 - MATCH (p1:Person),(p2:Person), p = shortestPath((p1)-[*]-(p2))
 - WHERE p1.name = "Kevin Bacon" AND p2.name <> "Kevin Bacon"
 - WITH length(p) AS length
 - RETURN max(length) AS maxLengt

- Q8: What is the largest shortest-path distance between any two persons?
- Answer
 - MATCH (p1:Person),(p2:Person), p = shortestPath((p1)-[*]-(p2))
 - WHERE p1 <> p2
 - WITH length(p) AS length
 - RETURN max(length) AS maxLength
- Q9: What is the distribution of pairwise distances (i.e., for distance 1, 2, 3, ..., how many pairs of persons have that distance from each other)?
- Answer
 - MATCH (p1:Person),(p2:Person), p = shortestPath((p1)-[*]-(p2))
 - WHERE p1 <> p2
 - WITH p1, length(p) AS length
 - RETURN length, count(*) AS count
 - ORDER BY length

- Q10: In the co-actor graph, which 10 persons have the shortest average distance to any other person he/she is connected to?
- Answer
 - MATCH (p1:Person), (p2:Person), p = shortestPath((p1)-[:ACTED_IN*]-(p2))
 - WHERE p1 <> p2
 - WITH p1, p
 - RETURN p1, avg(length(p)) AS avgLength
 - ORDER BY avgLength
 - LIMIT 10
- Q11: How many persons have persons co-acted with on average through distance 1, 2, 3, ...?
- Answer
 - MATCH (p1:Person), (p2:Person), p = shortestPath((p1)-[:ACTED_IN*]-(p2))
 - WHERE p1 <> p2
 - WITH p1, length(p) AS len
 - WITH p1, len, count(*) AS count
 - RETURN len / 2 as dist, avg(count)
 - ORDER BY dist

PY2NEO Example



- pip install py2neo

```
In [16]: from py2neo import Graph, Node, Relationship
g = Graph()
tx = g.begin()
a = Node("Person", name="Alice")
tx.create(a)
b = Node("Person", name="Bob")
ab = Relationship(a, "KNOWS", b)
tx.create(ab)
tx.commit()
g.exists(ab)
```

```
Out[16]: True
```

```
In [25]: g.run("MATCH (a:Person) RETURN a.name, a.born LIMIT 10").data()
```

```
Out[25]: [{'a.born': 1964, 'a.name': 'Keanu Reeves'},
{'a.born': 1967, 'a.name': 'Carrie-Anne Moss'},
{'a.born': 1961, 'a.name': 'Laurence Fishburne'},
{'a.born': 1960, 'a.name': 'Hugo Weaving'},
{'a.born': 1967, 'a.name': 'Andy Wachowski'},
{'a.born': 1965, 'a.name': 'Lana Wachowski'},
{'a.born': 1952, 'a.name': 'Joel Silver'},
{'a.born': 1978, 'a.name': 'Emil Eifrem'},
{'a.born': 1975, 'a.name': 'Charlize Theron'},
{'a.born': 1940, 'a.name': 'Al Pacino'}]
```

Note: I had this problem too. In my case I was looking at the [py2neo v2](#) documentation but on my machine was installed [py2neo v3](#). You should check your **py2neo** version and replace `.cyper({query})` with `.run({query})`

- https://neo4j.com/developer/example-data/#_neo4j_graph_gists
- <https://neo4j.com/blog/import-10m-stack-overflow-questions/>

- Occorre decidere come modellare i dati ed effettuare pre-processing sui dati di partenza.
- Import from Relational DBMS
- Import from MongoDB
- Import data in Neo4j from CSV
 - <https://neo4j.com/developer/guide-import-csv/>
 - <https://neo4j.com/graphacademy/training-intro-40/17-using-load-csv-import/>
 - <https://neo4j.com/docs/cypher-manual/current/clauses/load-csv/#load-csv-importing-large-amounts-of-data>
 - <https://neo4j.com/developer/guide-importing-data-and-etl/>

Dataset – Tweet US Airline (MongoDB e Neo4J)



- <https://www.kaggle.com/crowdflower/twitter-airline-sentiment>
- Quanti tweet per ciascun utente?
- Quanti retweet per ciascuna compagnia aerea?
- TagCloud dei termini;
- Quale è la compagnia con il maggior numero di tweet con sentiment negativa?
- Distribuzione della sentiment per ciascuna compagnia aerea;
- Distribuzione del sentiment negativo per ciascuna motivazione;
- Autori, in ordine alfabetico, che hanno ricevuto più di 10 retweet;
- Data in input una compagnia aerea, mostrare:
 - la tagCloud dei termini utilizzati in caso di sentiment negativa;
 - la tagCloud dei termini utilizzati in caso di sentiment positiva.

Esercitazione Neo4j4



- Supponiamo di avere un grafo composto da studenti, corsi, progetti, aule e relazioni tra essi. I nodi e le relazioni sono specificati in Cypher come indicato di seguito.
- Specificare le query delle slide seguenti.

Nodes

```
CREATE (s1: Student {studentID: "1", lastName: "Doe", firstName: "Ana", middleName: "Maria"})
CREATE (s2: Student {studentID: "2", lastName: "Ung", firstName: "Peter", middleName: "John"})
CREATE (s3: Student {studentID: "3", lastName: "Doe", firstName: "John"})
CREATE (s4: Student {studentID: "4", lastName: "Berre", firstName: "Stine"})
CREATE (s5: Student {studentID: "5", lastName: "Travolta", firstName: "John"})
CREATE (c1: Course {courseNr: "1", courseName: "Databases"})
CREATE (c2: Course {courseNr: "2", courseName: "Programming"})
CREATE (c3: Course {courseNr: "3", courseName: "Graphics"})
CREATE (p1: Project {projectNr: "34", projectName: "eCommerce database"})
CREATE (p2: Project {projectNr: "24", projectName: "eCommerce website"})
CREATE (p3: Project {projectNr: "13", projectName: "User interface"})
CREATE (p4: Project {projectNr: "26", projectName: "Reporting"})
CREATE (r1: Room {roomName: "Pascal"})
CREATE (r2: Room {roomName: "Seminar C"})
CREATE (r3: Room {roomName: "Alpha"})
CREATE (r4: Room {roomName: "Beta"})
```

Relationships

```
CREATE (c1) -[:TAKESPLACEIN] -> (r1)
CREATE (c1) -[:TAKESPLACEIN] -> (r3)
CREATE (c1) -[:TAKESPLACEIN] -> (r4)
CREATE (c2) -[:TAKESPLACEIN] -> (r2)
CREATE (s1) -[:ENROLLEDIN] -> (c1)
CREATE (s2) -[:ENROLLEDIN] -> (c1)
CREATE (s3) -[:ENROLLEDIN] -> (c2)
CREATE (s4) -[:ENROLLEDIN] -> (c1)
CREATE (s1) -[:WORKSON {hours: "1"}] -> (p1)
CREATE (s1) -[:WORKSON {hours: "2"}] -> (p2)
CREATE (s2) -[:WORKSON {hours: "3"}] -> (p1)
CREATE (s2) -[:WORKSON {hours: "4"}] -> (p2)
CREATE (s2) -[:WORKSON {hours: "1"}] -> (p3)
CREATE (s2) -[:WORKSON {hours: "1"}] -> (p4)
CREATE (s3) -[:WORKSON {hours: "1"}] -> (p1)
CREATE (s3) -[:WORKSON {hours: "2"}] -> (p2)
CREATE (s3) -[:WORKSON {hours: "3"}] -> (p4)
```